A Minimal Self-Referential Graph Formalism for Internal Semantic and Temporal Structure

University of London

MSc Computer Science

Project Proposal

Author: Shahzada Saeed

Word Count: 2,476

13 July 2026

Abstract

This proposal outlines a combined theoretical and practical investigation into whether a discrete computational system can internalise its own interpretation. The project will define a minimal self-referential graph formalism in which rewrite rules are encoded as subgraphs of the very graph they transform, "meaning" is characterised as fixed points of internal rewriting, and temporal order emerges as a partial order over rewrite-event dependencies, without externally assigned semantic labels. Formal results will be established using fixed-point theory, confluence and termination criteria from rewriting theory, and event-structure models from concurrency theory. Co-equal with the theory, the project will deliver a software artefact: an interactive workbench comprising a Rust rewrite engine, parser, scheduling strategies, causal-poset instrumentation, a property-based test suite, and a browser-based visualiser. The work contributes an executable reference semantics for self-referential graph rewriting to the foundations and semantics subfield of theoretical computer science.

Contents

1. Introduction 3
2. Problem Statement 4
3. Research Questions 5
4. Research Aims 6
5. Literature Review 7
   1.1. Semantics, Reflection, and Self-Reference 7
   1.2. Graph Rewriting 7
   1.3. Fixed Points, Convergence and Semantic Stability 8
   1.4. Causality, Event Structures and Emergent Time 8
   1.5. Software Precedents 8
6. Formal Approach 8
7. Methodology 9
8. Project Plan and Timeline 10
9. Expected Outcomes 10
10. Bibliography 12

1. Introduction

Standard models of computation separate syntax from semantics. Operational and denotational semantics rely on an externally specified meta-theory to confer meaning on otherwise inert symbols, and knowledge-graph technologies similarly presuppose predefined ontologies and labels. This separation is well understood, but it raises a foundational question: can a discrete computational system internalise its own interpretation, such that semantic stability and temporal ordering arise from the system's own structural dynamics rather than from an external semantic layer?

Several established formal devices make the question tractable. Graph rewriting treats computation as the transformation of structure (Heckel, 2006); reflective towers of interpreters embed evaluators in the languages they evaluate (Smith, 1984; Amin and Rompf, 2018); event structures and causal partial orders capture temporal order without explicit clocks (Winskel, 1987); and Lamport's happens-before relation shows that ordering can emerge from dependency alone (Lamport, 1978). The pieces exist, but no single minimal formalism unifies them.

This project defines and analyses such a formalism and provides a software deliverable: an interactive workbench that serves as an executable reference semantics, with instrumentation that makes the formal results inspectable, testable, and demonstrable.

2. Problem Statement

Self-reference is computationally possible but difficult to control. Kleene's recursion theorem establishes that programs can obtain fixed points of transformations over program descriptions (Kleene, 1952), while reflective languages show that object- and meta-level operations can be made available within a running system. However, reflection commonly retains staging, typing or other stratification to prevent semantic paradox and uncontrolled regress. The reFLect language, for example, combines quotation with evaluation but demonstrates why unrestricted self-description requires careful type discipline (Grundy, Melham and O'Leary, 2006).

Graph rewriting offers an appropriate substrate because rules operate directly on structure, and term graphs represent sharing, cycles and recursive definitions more naturally than trees

(Barendregt et al., 1987; Bahr, 2012). Existing systems nevertheless tend to keep transformation rules external to the host graph, or permit limited self-description without integrating fixed-point stability and causal histories. Graph automata with metanodes provide a notable exception by embedding descriptions of local transformations, but their emphasis is self-reproduction rather than fixed-point semantics or causal event models (Tomita, Kurokawa and Murata, 2002).

The gap is the absence of a deliberately small, executable account in which data and active rule representations share one graph domain, while rewrite-event dependencies are derived within the same operational account. "Internal" will mean that rule content, activation and modification are represented in the object graph; it will not mean that matching and transition lack an externally specified semantics.

The project uses "meaning" in a restricted technical sense. An internally semantic configuration is an equivalence class of complete graph states, including active rule subgraphs, that is invariant under a designated interpretation relation, or a fixed point of a corresponding operator where suitable order-theoretic conditions hold. This does not claim to solve linguistic meaning, consciousness or symbol grounding. The problem is to determine whether this stability notion can be non-trivial, when it converges, and how much computation the formalism can express.

3. Research Questions

The thesis is that there exists a minimal self-referential graph formalism in which (i) interpretation is realised as internal graph rewriting, (ii) "meaning" can be defined as fixed points of that rewriting, and (iii) a temporal partial order emerges from rule-application dependencies, without appeal to externally assigned semantic labels. Five questions structure the investigation:

RQ1. What is a minimal node-and-edge formalism in which a graph can encode its own rewrite rules while avoiding unbounded meta-regress?

RQ2. Under what structural and operational conditions does internal rewriting terminate, converge, or reach a stable equivalence class?

RQ3. Which fixed-point, normal-form and observational criteria distinguish semantic stability, persistent oscillation or divergence, and trivial collapse?

RQ4. How can dependency between rewrite events induce a causal partial order, and how does that order relate to event structures and established models of concurrent computation?

RQ5. Where does the formalism sit, in expressive power, relative to the untyped lambda calculus and established graph rewriting systems?

4. Research Aims

Five research aims correspond to these questions. First, define a finite typed directed graph model, a small vocabulary of structural and meta-level relations, a graph-resident rule representation, and a guardedness or ranking condition that permits rule-on-rule transformation without infinite regress. Primitive types will specify operational roles rather than domain meanings. Minimality will be assessed through eliminability: a primitive is unnecessary if its removal does not reduce the ability to encode graph-resident rewriting, guarded self-modification, the selected examples or the expressiveness result.

SSecond, it will specify a small-step structural operational semantics for matching, rule activation, replacement and self-modification. Rules will be identifiable subgraphs rather than opaque host-language functions. Rule-on-rule transformations will be admissible only where the resulting configuration preserves the guardedness or rank invariant.

Third, establish rigorous results for a restricted core. The analysis will distinguish normal forms from fixed points, apply Knaster-Tarski only where monotonicity and a suitable complete lattice are demonstrated, use Kleene's recursion theorem to characterise computable self-reference rather than convergence, and apply termination, local-confluence and critical-pair reasoning where appropriate (Tarski, 1955; Newman, 1942; Baader and Nipkow, 1998). For non-deterministic rewriting, stability will be formulated relationally or through a powerset transformer rather than assuming one deterministic operator.

Fourth, define rewrite events by their consumed, preserved and produced graph elements, derive enabling and dependency relations, and prove when the transitive closure forms a partial order. The result will be compared with Lamport's happened-before relation and event

structures, including the distinction between genuine causal dependence and independent events merely serialised by a scheduler (Nielsen, Plotkin and Winskel, 1981).

Fifth, establish expressiveness through explicit encodings. A specified non-trivial fragment of the untyped lambda calculus will be translated into the proposed graphs, with a reduction-correspondence argument for supported constructs. A formal relationship to standard term-graph rewriting will locate sharing, recursion and self-modifying rules relative to an established model, and strict limits will be stated explicitly.

The engineering objective is to implement the workbench described in Section 6. Implementation will influence the formal definitions, automated tests will search for counterexamples, and exported traces and benchmarks will form evidence in the dissertation's evaluation.

5. Literature Review

1.1. Semantics, Reflection, and Self-Reference

Classical semantics places the defining machinery outside the system being defined. Reynolds (1972) showed that definitional interpreters can inherit properties of their metalanguage, warning that an "internal semantics" project may merely conceal an external one. Smith (1984) established computational reflection and towers of interpreters as a serious research topic, while Amin and Rompf (2018) showed that such towers can be collapsed under disciplined staging. The reFLect experience demonstrates the hazards of unstratified self-description (Grundy, Melham and O'Leary, 2006). In contrast, Kleene's recursion theorem shows that self-reference itself is not exceptional. The unresolved issue is semantic control, motivating guardedness and the deliberately narrow definition of internal meaning.

1.2. Graph Rewriting

Graph transformation provides a precise account of rule-governed structural change (Heckel, 2006). General algebraic approaches offer powerful matching and gluing theory, but may obscure the intended minimal core. Term-graph rewriting is more directly relevant because sharing and cycles support compact representations of recursive computation (Barendregt et al., 1987; Bahr, 2012). Work on the lambda calculus with letrec shows that graph semantics can capture maximal sharing while preserving unfolding behaviour (Grabmayer and Rochel,

2014). Interaction nets demonstrate that local graph reduction can be strongly confluent and computationally universal (Lafont, 1990), but their restrictive active-pair discipline is poorly suited to mutable rules represented as inspectable subgraphs. These results favour a small first-order graph core informed by term graphs.

1.3. Fixed Points, Convergence and Semantic Stability

Knaster-Tarski establishes fixed points for monotone operators on complete lattices (Tarski, 1955), while rewriting theory supplies operational criteria for convergence and uniqueness. Newman's lemma connects termination and local confluence to confluence, and critical-pair analysis identifies overlaps that threaten unique normal forms (Newman, 1942; Baader and Nipkow, 1998). Bahr (2012) shows that convergence in term graphs can be analysed order-theoretically and metrically. The project will distinguish stable semantic configurations, operational normal forms, divergence or oscillation, and trivial collapse, where convergence occurs only because relevant structure has been erased or homogenised.

1.4. Causality, Event Structures and Emergent Time

Lamport (1978) derives a partial order from causal dependency without requiring a universal clock. Event structures enrich this through conflict and enabling (Nielsen, Plotkin and Winskel, 1981; Winskel, 1987), while graph-transformation research shows that non-linear rules may require event models richer than prime event structures (Baldan, Corradini and Gadducci, 2017). Execution order is not automatically causal order: dependency must be reconstructed from resource production, consumption, preservation and rule versions, excluding order introduced solely by a sequential scheduler.

1.5. Software Precedents

Existing tools demonstrate that software can be a research instrument in formal semantics. GP 2 provides an executable graph-programming language grounded in operational semantics (Plump, 2012); PORGY supports strategy-driven exploration of rewriting and derivation histories (Fernandez, Kirchner and Pinaud, 2019); and Quantomatic supports computer-assisted diagrammatic rewriting (Kissinger and Zamdzhiev, 2015). Property-based testing turns consequences of formal claims into executable properties over generated examples (Claessen and Hughes, 2000). The remaining gap is an instrument focused on graph-resident rules, guarded self-modification, stability analysis and an execution-derived causal order.

6. Formal Approach

The initial candidate core is a typed directed graph with a small vocabulary of node kinds, such as constructor, variable, rule-head and guard, and edge kinds representing structural, binding and meta-level relations. A rewrite rule is a subgraph pairing a left-hand pattern with a right-hand replacement; a distinguished relation marks rule subgraphs as active. A small-step semantics defines matching, applicability and replacement. Each event records the graph elements it reads, preserves, consumes and produces, together with the active rule version. These records generate an immediate-dependency DAG whose reachability relation defines the strict causal partial order.

The workbench will use a Rust engine and a TypeScript browser interface. Rust's algebraic data types and exhaustive matching suit a semantics-oriented implementation, while ownership and memory safety reduce risks in destructive graph updates. The engine will include a concrete syntax and parser, graph validation, rule matching and replacement, deterministic and seeded non-deterministic schedulers, causal instrumentation, trace export and invariant checking. It will compile to WebAssembly or expose a small local interface to a TypeScript visualiser using D3 or a suitable equivalent. The interface will display the evolving graph and causal DAG side by side and support step-through execution, rule breakpoints and trace inspection.

The workbench will include unit tests and property-based tests encoding executable consequences of selected lemmas over generated finite graphs. Its example library will cover graph-resident arithmetic rules, guarded rule self-modification, a non-trivial stable configuration, divergence or oscillation, trivial collapse, independent concurrent rewrites and reduction of selected lambda-calculus terms.

7. Methodology

Theory and implementation will proceed together in two-week agile sprints. Each theoretical increment will be implemented promptly so that the executable system exposes omissions before extensive proof effort is invested. Formal work will follow a repeated pipeline: definition, worked example, conjecture, property-based falsification attempt and conventional mathematical proof. Tests will support but not replace proofs.

Software evaluation will not involve human participants. Correctness will be assessed by comparing generated traces with manually derived applications of the operational semantics; unit testing matching, replacement and graph validation; and using property-based tests to search for violations of well-formedness, guardedness and causal-order invariants. Causal instrumentation will be tested for acyclicity, dependency preservation and recognition of scheduler-independent concurrency. The lambda-calculus case study will assess reduction correspondence. Controlled benchmarks will measure execution time and memory use as graph size, rule count, branching and trace length increase. Seeded scheduling and exported traces will support reproducibility.

The main risk is over-ambition. This will be mitigated by fixing the restricted formal core early, making the specified fragment encoding the minimum expressiveness result, and implementing the visualiser as progressive enhancement over a testable trace-producing engine. The dissertation will be drafted throughout the project.

8. Project Plan and Timeline

Sprint	Dates	Theory focus	Software focus
1	13 Jul – 26 Jul	Literature consolidation; candidate primitives; guardedness and minimality criteria.	Rust workspace, CI, graph model and walking-skeleton interpreter.
2	27 Jul – 9 Aug	Small-step semantics; matching, replacement and guarded rule modification.	Concrete syntax, parser, rule-subgraph execution and arithmetic example.
3	10 Aug – 23 Aug	Fixed-point taxonomy; termination, confluence and restricted-core results.	Schedulers, invariant checks, property tests, stable and divergent examples.
4	24 Aug – 6 Sep	Rewrite-event dependencies; causal partial order; event-structure comparison.	Causal instrumentation, trace export, WASM bridge and visualiser MVP.
5	7 Sep – 20 Sep	Lambda-fragment encoding; reduction correspondence; term-graph comparison and limits.	Lambda examples, benchmarks, example library and visualiser refinement.
6	21 Sep – 1 Oct	Proof audit and theory freeze.	Code freeze, documentation, packaging, final evaluation and submission.

9. Expected Outcomes

The successful completion of this project will yield significant outcomes contributing to both academic knowledge and practical system development.

Technical Deliverables: A functioning interactive workbench comprising a Rust graph-rewriting engine, concrete syntax and parser, graph-resident rule execution, deterministic and non-deterministic schedulers, causal-dependency instrumentation, automated tests, and a browser-based visualiser. The software will include reproducible traces and examples covering arithmetic evaluation, rule self-modification, stable configurations, divergence, causal independence and lambda-calculus reduction.

Research Contributions: A precise specification and executable reference semantics for a restricted self-referential graph formalism. The research will identify conditions for controlled self-reference and semantic stability, distinguish fixed points, normal forms, divergence and trivial collapse, derive a partial order from rewrite-event dependencies, and establish expressiveness results through comparison with term-graph rewriting and a selected fragment of the untyped lambda calculus.

Academic Impact: The research contributes to programming-language semantics and graph rewriting through an internally represented rule system, to concurrency theory through a rewrite-derived causal order, and to formal software engineering by integrating mathematical proof, property-based falsification and executable semantics within one research artefact.

10. Bibliography

Amin, N. and Rompf, T. (2018) ‘Collapsing towers of interpreters’, Proceedings of the ACM on Programming Languages, 2(POPL), Article 52.

Baader, F. and Nipkow, T. (1998) Term Rewriting and All That. Cambridge: Cambridge University Press.

Bahr, P. (2012) ‘Infinitary term graph rewriting is simple, sound and complete’, in 23rd International Conference on Rewriting Techniques and Applications (RTA 2012). Dagstuhl: LIPIcs, pp. 69–84.

Baldan, P., Corradini, A. and Gadducci, F. (2017) ‘Domains and event structures for fusions’, in 32nd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2017). IEEE, pp. 1–12.

Barendregt, H.P. (1984) The Lambda Calculus: Its Syntax and Semantics. Revised edn. Amsterdam: North-Holland.

Barendregt, H.P., van Eekelen, M.C.J.D., Glauert, J.R.W., Kennaway, J.R., Plasmeijer, M.J. and Sleep, M.R. (1987) ‘Term graph rewriting’, in PARLE: Parallel Architectures and Languages Europe. Berlin: Springer (LNCS 259), pp. 141–158.

Bergstra, J.A. and Burgess, M. (2014) Promise Theory: Principles and Applications. Oslo: χtAxis Press.

Burgess, M. (2014) ‘Spacetimes with semantics: notes on the “meaning” of space, time and process’. arXiv:1411.5563.

Burgess, M. (2019) Smart Spacetime: How Information Challenges Our Ideas About Space, Time and Process. Oslo: χtAxis Press.

Claessen, K. and Hughes, J. (2000) ‘QuickCheck: a lightweight tool for random testing of Haskell programs’, in Proceedings of the 5th ACM SIGPLAN International Conference on Functional Programming (ICFP 2000). ACM, pp. 268–279.

Ehrig, H., Ehrig, K., Prange, U. and Taentzer, G. (2006) Fundamentals of Algebraic Graph Transformation. Berlin: Springer.

Fernández, M., Kirchner, H. and Pinaud, B. (2019) 'Strategic port graph rewriting: an interactive modelling framework', Mathematical Structures in Computer Science, 29(5), pp. 615–662.

Fontana, W. and Buss, L.W. (1994) '"The arrival of the fittest": toward a theory of biological organization', Bulletin of Mathematical Biology, 56(1), pp. 1–64.

Grabmayer, C. and Rochel, J. (2014) 'Maximal sharing in the lambda calculus with letrec', in Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP 2014). ACM, pp. 67–80.

Grundy, J., Melham, T. and O'Leary, J. (2006) 'A reflective functional language for hardware design and theorem proving', Journal of Functional Programming, 16(2), pp. 157–196.

Heckel, R. (2006) 'Graph transformation in a nutshell', Electronic Notes in Theoretical Computer Science, 148(1), pp. 187–198.

Hofstadter, D.R. (1979) Gödel, Escher, Bach: An Eternal Golden Braid. New York: Basic Books.

Kampis, G. (1991) Self-Modifying Systems in Biology and Cognitive Science: A New Framework for Dynamics, Information and Complexity. Oxford: Pergamon Press.

Kissinger, A. and Zamdzhiev, V. (2015) 'Quantomatic: a proof assistant for diagrammatic reasoning', in Automated Deduction — CADE-25. Cham: Springer (LNCS 9195), pp. 326–336.

Kleene, S.C. (1952) Introduction to Metamathematics. Amsterdam: North-Holland.

Lafont, Y. (1990) 'Interaction nets', in Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '90). ACM, pp. 95–108.

Lamport, L. (1978) 'Time, clocks, and the ordering of events in a distributed system', Communications of the ACM, 21(7), pp. 558–565.

Maturana, H.R. and Varela, F.J. (1980) Autopoiesis and Cognition: The Realization of the Living. Dordrecht: D. Reidel.

Milner, R. (2009) The Space and Motion of Communicating Agents. Cambridge: Cambridge University Press.

Newman, M.H.A. (1942) ‘On theories with a combinatorial definition of “equivalence”’, Annals of Mathematics, 43(2), pp. 223–243.

Nielsen, M., Plotkin, G. and Winskel, G. (1981) ‘Petri nets, event structures and domains, part I’, Theoretical Computer Science, 13(1), pp. 85–108.

Plotkin, G.D. (2004) ‘A structural approach to operational semantics’, Journal of Logic and Algebraic Programming, 60–61, pp. 17–139.

Plump, D. (2012) ‘The design of GP 2’, in Proceedings of the 10th International Workshop on Reduction Strategies in Rewriting and Programming (WRS 2011). EPTCS 82, pp. 1–16.

Reynolds, J.C. (1972) ‘Definitional interpreters for higher-order programming languages’, in Proceedings of the ACM Annual Conference. ACM, pp. 717–740.

Smith, B.C. (1984) ‘Reflection and semantics in Lisp’, in Proceedings of the 11th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL ’84). ACM, pp. 23–35.

Tarski, A. (1955) ‘A lattice-theoretical fixpoint theorem and its applications’, Pacific Journal of Mathematics, 5(2), pp. 285–309.

Tomita, K., Kurokawa, H. and Murata, S. (2002) ‘Graph automata: natural expression of self-reproduction’, Physica D: Nonlinear Phenomena, 171(4), pp. 197–210.

Winskel, G. (1987) ‘Event structures’, in Petri Nets: Applications and Relationships to Other Models of Concurrency. Berlin: Springer (LNCS 255), pp. 325–392.

Wolfram, S. (2020) ‘A class of models with the potential to represent fundamental physics’, Complex Systems, 29(2), pp. 107–536.